TEMCO LEARN

Temco Platform

`temco-workbench` · built Nov 10, 2026

`temco-workbench`

The application's own surfaces. It is the only module that is not a domain: it owns the **Library** (the index of what every module has), the **dashboard**, the **needs page**, and the three layout decisions that come with them — which side panel and which tab open at startup, and which sidebar section comes forward when the project changes.

Folder	`extensions/temco-workbench`
Extension id	`temcocontrols.temco-workbench`
Namespace	`temcoWorkbench`
State	active
View	**Library** (`temcoWorkbench.library`), contributed into the `temco` activity-bar container that the documentation module declares
Pages	`temcoWorkbench.overview` (the dashboard), `temcoWorkbench.activity` (what the platform recorded), `temcoWorkbench.openNeeds` (what wants a person), `temcoWorkbench.welcome` (the page a new window opens on)
Setting	`temcoWorkbench.start.open` — `welcome` · `project` · `page` · `home` · `none` (default `welcome`) · `temcoWorkbench.start.panel` — the left panel's item revealed at startup (default `temcoDashboard`) · `temcoWorkbench.updateChannel` and the `branding` · `ai` · `publishing` keys the Settings pages write
Requires	`temco-store` , `temco-ui` , `temco-repositories` , `temco-docs`
Required by	`temco-docs` — nothing; the dependency runs one way, and the workbench reads the documentation module's model

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation: builds the three surfaces, wires their signals, and decides the first tab
`src/sidebar.ts`	the side-bar groups that are not a data view — Platform, Needs Work, Site, Delivery, Profile, Learn, Support — registered here because a container cannot declare its order in the activity bar
`src/dashboard.ts`	the dashboard's two pages: **Overview** (what the platform holds, what is waiting, the pipeline, the activity) and **Recent Activity**
`src/helpPages.ts`	Welcome, Help, Keyboard Shortcuts, Logs, Report an Issue, About — and the one preference Welcome carries
`src/settingsPages.ts`	the five settings pages (General, Modules, Branding, AI Runtime, Publishing). Every row is a declared setting, so a write lands in `settings.json` and the window obeys it at once; the page reads the configuration back and never keeps a value of its own
`src/needsPage.ts`	what wants a person, gathered from every module into one list with the reason and where the fix is
`src/learnPages.ts`	Temco Learn as a reader sees it: Home, T3000, an article, Downloads, Learning Paths
`src/surface.ts`	the page view types
`src/docs.ts` · `src/store.ts` · `src/ui.ts` · `src/repositories.ts` · `src/aiModule.ts`	what it reads of the documentation module, the store, the UI rules, the repository scripts and the AI — each in one file, because an extension cannot import another extension's sources

Commands

`temcoWorkbench.home` (the dashboard) · `temcoWorkbench.openNeeds` (the needs page) · `temcoWorkbench.welcome` (the welcome page) · `temcoWorkbench.toggleWelcomeStartup` (the checkbox in its footer, which writes `temcoWorkbench.start.open`) · `temcoWorkbench.settings` (the settings page) · `temcoWorkbench.setStartOpen` (the one command a row on it runs) · `temcoWorkbench.help` (the help page). Everything the Library's rows and menus do is another module's command — the menus are here, the work is not.

What the pages refuse to invent

The mock has surfaces whose content this product cannot honestly produce, and none of them were copied across:

Mock page	Here
Help › Keyboard Shortcuts, Help › Logs	Not built. VS Code publishes neither list to an extension, and a page that guessed would be a page that lies. The Help page carries what is real instead: the build, the module register, the platform string, and the issue tracker.
Workspaces	Removed. A workspace was the mock's folder-above-the-project level, drawn over a fixed sample (`T3000` , `T3-BACnet` , …) with counts nothing recorded, and a “create” that wrote only into the window. The product's unit is the **project**, which the catalog holds; its sources are the folders, and New Project is the wizard the sample stood in for.
Devices, I/O Studio, BACnet, Network, SQL Studio	Not built. Those modules are reserved — a page over a module that does not exist would be a mock inside the product.

One home: Welcome, the VS Code way

A window that opens on a page you did not ask for has to earn it, so the welcome page carries only what a person can act on: six ways in (each a button running another module's command), three walkthroughs that list their steps instead of claiming progress, and a footer whose single checkbox writes the setting that decides the first tab. There is no picture of the product on it.

The arrangement is VS Code's, deliberately:

- **Welcome is the first tab.** `start.open` defaults to `welcome` , so a launch answers "what is this and where do I begin" before it shows any work.
- **The page owns the switch.** Its footer checkbox *Show the welcome page at startup* writes `temcoWorkbench.start.open` — there is no second place to turn it off, and the checkbox and the setting cannot disagree.
- **Help ▸ Welcome brings it back**, as does the palette, every time.
- **The Dashboard is not a second home.** It is the activity-bar container you go to *after* Welcome — Overview (what the platform holds and what is waiting), Recent Activity, Needs Work. VS Code's equivalent is Explorer: state you click into, never a page that opens itself beside the welcome page.

The two only ever fought because either could be the first tab. Now one is the default and the other is a click, and each has one question: Welcome asks *where do I start*, the Overview asks *what is in it and what needs me*. With the checkbox off the chain never ends at the welcome page — `project` → the page you were last on → the overview — so turning it off means what it says; an empty window gets the overview, which has its own empty state and a *New Project* button. **Coverage & Health** is gone as a third dashboard page: the documentation module draws that report from the same registry with the same figures, so the Overview's Knowledge card links to it instead of duplicating it.

Where a settings page writes

Every row on the five Settings pages is a declared setting, so everything a page decides is in the profile's `settings.json`:

Page	Keys
General	`workbench.colorTheme` · `temcoWorkbench.start.panel` · `workbench.breadcrumbs.enabled` · `telemetry.telemetryLevel` · `temcoWorkbench.updateChannel`
Modules	none — the rows describe what is loaded, and every button runs another module's command
Branding	`temcoWorkbench.branding.productName` · `.accent` — the name the pages draw, and the accent written into `workbench.colorCustomizations`
AI Runtime	`temcoAi.model` · `.runtime` · `.modelFolder` · `.allowUnpinned` · `.idleMinutes` · `.serverPort` · `.serverIdleMinutes` · `.platformApi` — the AI module's own keys, so the page writes exactly what that module reads
Publishing	`temcoWorkbench.publishing.site` · `.path` · `.channel` · `.website` · `.pdf` · `.offlineHelp` · `.searchIndex` · `.aiIndex`

Two facts are not settings and are not written there: **when a row last changed** and **what Test Connection last read back**. Both are kept in the profile's own state rather than in the file, because they record what happened rather than what a person chose. `Restore Defaults` removes the value instead of writing a copy of today's default, so the manifest stays the only place a default is stated.

Boundary rules

- **The Library is an index, not a domain.** It shows what the store recorded and what the documentation module published; it owns no data of its own and keeps no copy.
- **A row's action belongs to the module that owns the thing.** *Read a source* is `temcoRepositories.readSource`, *Build the project* is `temcoProjects.buildProject`, a page's *Rename* is `temcoDocs.renamePage`. The Library names them; it never re-implements one.
- **Nothing here writes.** The only state it changes is the project being worked on, and that is the store's own command.
- **The AI is optional for it too.** The Library's AI row appears only when that module is installed — asked, never assumed.
- **The Workspace is not its view.** The tree of pages is the documentation module's own view; the workbench only *brings it forward* by id when the project changes.